Mahidol University
Faculty of Information
and Communication Technology

Command Line Operations

ITCS 254 | Lecture 4

Ittipon (Art) Rassameeroj



Agenda



Hard Links and Soft Links



Working with Files



Searching for Files

Hard Link

- The `ln` utility is used to create hard links and (with the `-s` option) soft links, also known as symbolic links or symlinks.
- These two kinds of links are very useful in UNIX-based OSs.
- Hard links are very useful, and they save space.
- If you remove either file1 or file2 in the example, the inode object (and the remaining file name) will remain, which might be undesirable, as it may lead to subtle errors later if you recreate a file of that name.
- If you edit one of the files, exactly what happens depends on your editor.
- Most editors will retain the link by default, but it is possible that modifying one of the names may break the link and result in the creation of two objects.

Soft (Symbolic) Links

- Symbolic links take no extra space on the filesystem (unless their names are very long).
- They are extremely convenient, as they can easily be modified to point to different places. An easy way to create a shortcut from your home directory to long pathnames is to create a symbolic link.
- Unlike hard links, soft links can point to objects even on different filesystems, partitions, and/or disks and other media, which may or may not be currently available or even exist.
- In the case where the link does not point to a currently available or existing object, you obtain a dangling link.

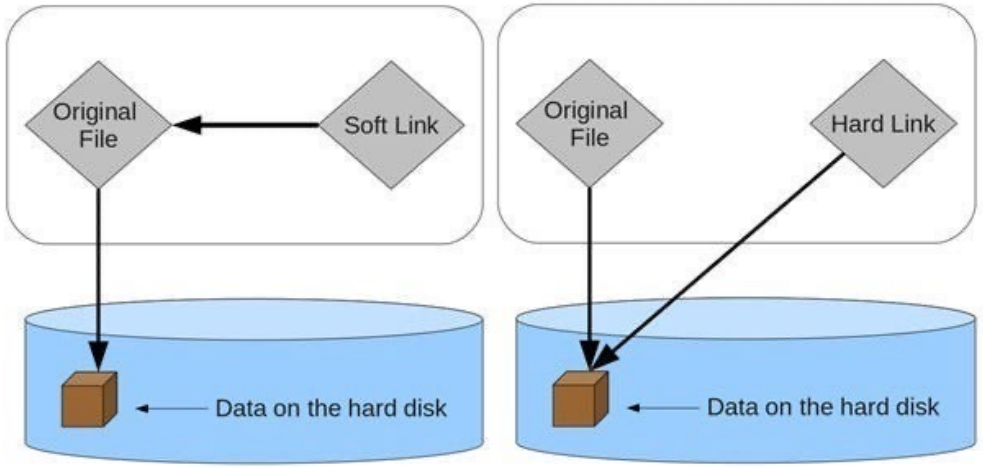
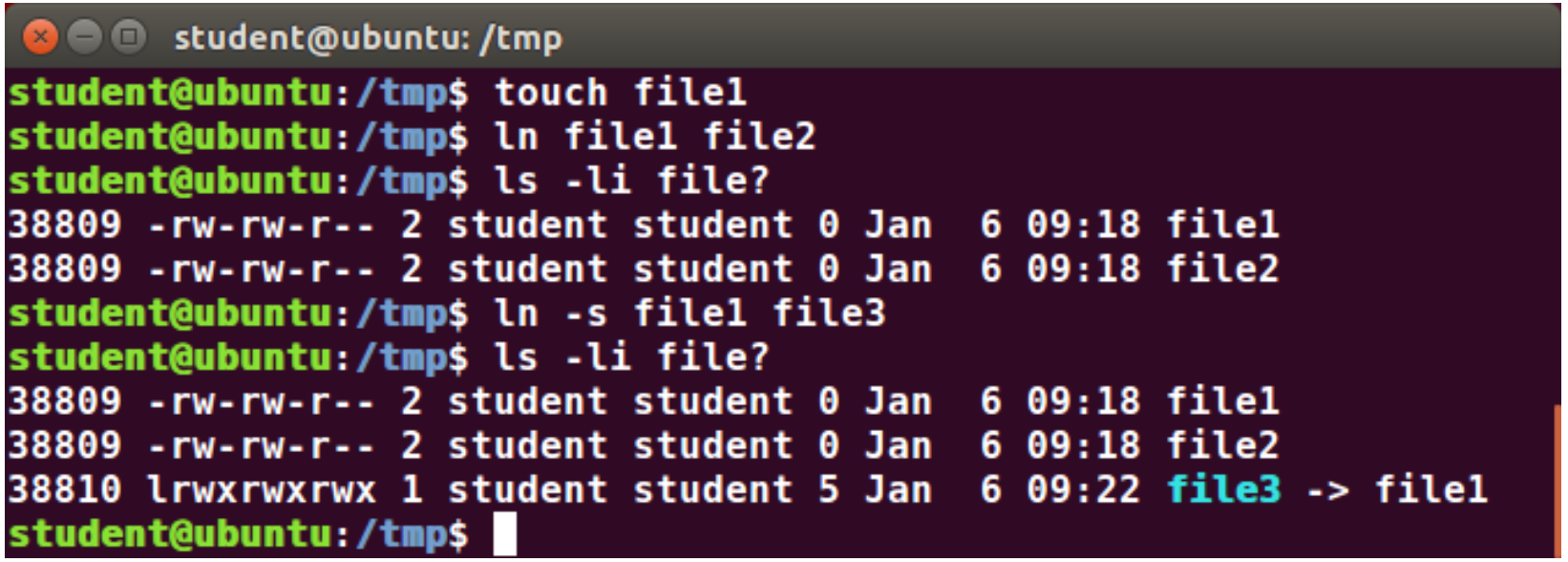


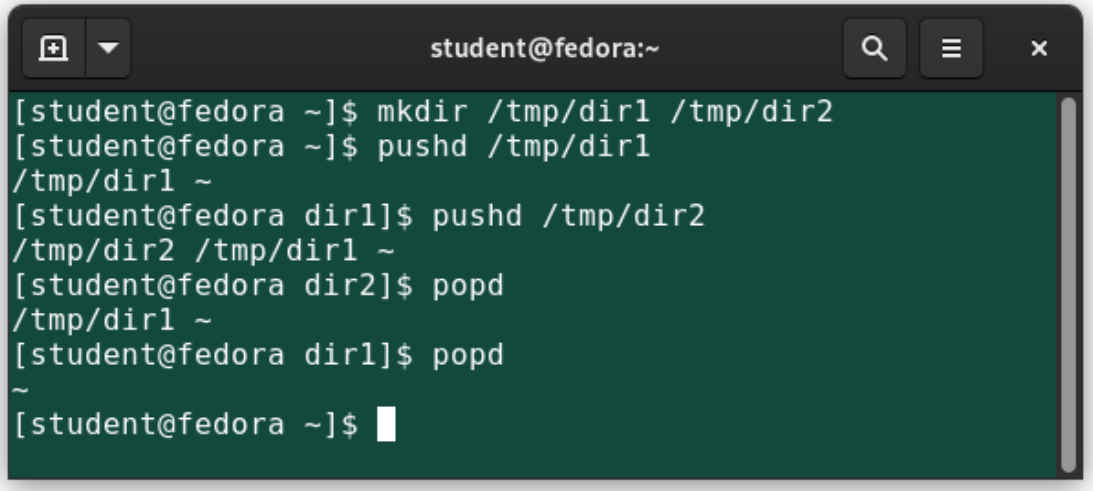
Image source: https://medium.com/@The_Anshuman/soft-link-and-hard-link-in-linux-3c0d7c1341c6

Hard Links Vs Soft Links



Navigating Through Directory History

- The `cd` command remembers where you were last, and lets you get back there with `cd -`.
- For remembering more than just the last directory visited, use `pushd` to change the directory instead of `cd`; this pushes your starting directory onto a list.
- Using `popd` will then send you back to those directories, walking in reverse order (the most recent directory will be the first one retrieved with `popd`).
- The list of directories is displayed with the `dirs` command.



Working with Files: Viewing Files

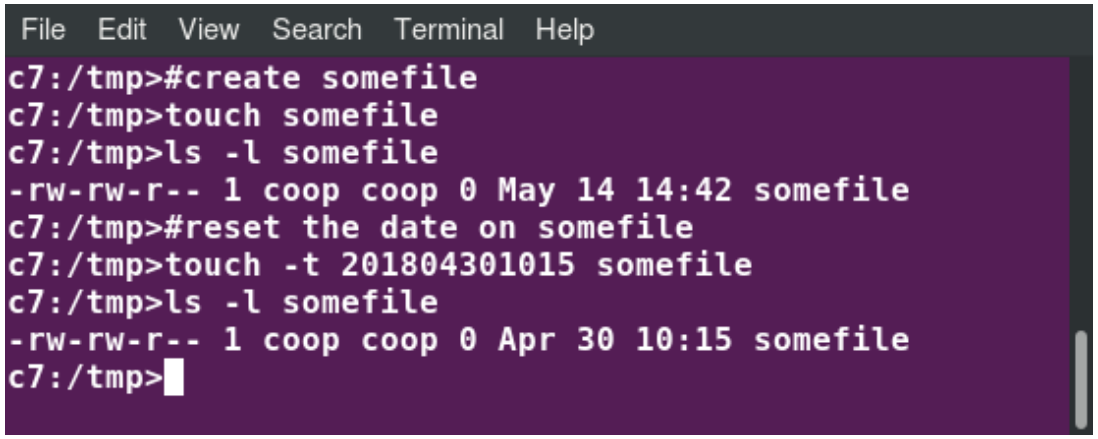
Command	Description
`cat` *filename*	Used for viewing files that are not very long; it does not provide any scroll-back.
`tac` *filename*	Used to look at a file backwards, starting with the last line.
`less` *filename* `more` *filename*	Used to view larger files because it is a paging program. It pauses at each screen full of text, provides scroll-back capabilities, and lets you search and navigate within the file. NOTE: Use `/` to search for a pattern in the forward direction and `?` for a pattern in the backward direction. An older program named more is still used, but has fewer capabilities: "less is more".
`tail` *filename*	Used to print the last 10 lines of a file by default. You can change the number of lines by doing -n 15 or just -15 if you wanted to look at the last 15 lines instead of the default.
`head` *filename*	The opposite of tail; by default, it prints the first 10 lines of a file.
`wc` *filename*	Display the total number of lines

Working with Files: `touch`

- `touch` is often used to set or update the access, change, and modify times of files.
- By default, it resets a file's timestamp to match the current time.
- However, you can also create an empty file using touch:

```
$ touch <filename>

$ touch -t 12091600 myfile
```

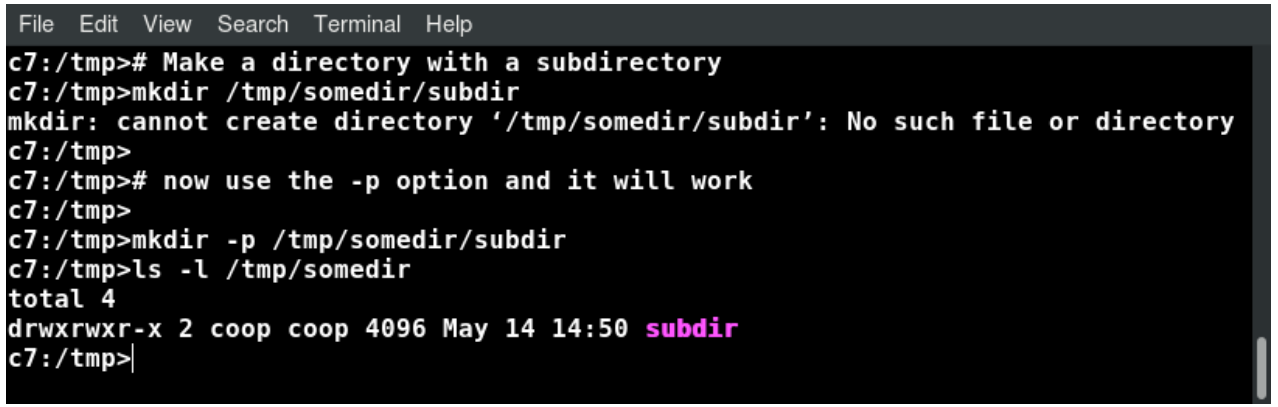


Moving, Renaming, and Copying a File

Command	Pattern	Usage
`mv`	`mv current_file new_file` `mv source directory_path`	• Rename a file, • move a file to another location, while possibly changing its name at the same time.
`cp`	`cp source_file target_file`	• Copy from a source file to another file
`rm`	`rm file`	• Remove a file
`rm -f`	`rm -f file`	• Forcefully remove a file
`rm -i`	`rm -i file`	• Interactively remove a file

Making, Moving, Renaming, and Copying a Directory

Command	Pattern	Description
`mkdir`	`mkdir` *`directory`*	Make a new directory
`mv`	`mv` *`current_dir new_dir`*	Rename a directory
`cp -R`	`cp -R` *`source_dir target_dir`*	Copy a directory to another directory
`rmdir`	`rmdir` *`directory`*	Remove an empty directory
`rm -r`	`rm -r` *`directory`*	Remove a directory
`rm -rf`	`rm -rf` *`directory`*	Forcefully remove a directory



Standard File Streams and I/O Redirection

Name	Symbolic Name	Value	Example
Standard input	`stdin`	0	Keyboard
Standard output	`stdout`	1	Terminal
Standard error	`stderr`	2	Log file

```
$ program < input-file

$ program > output-file

$ program < input-file > output-file

$ program 2> error-file

$ program 1> output-file

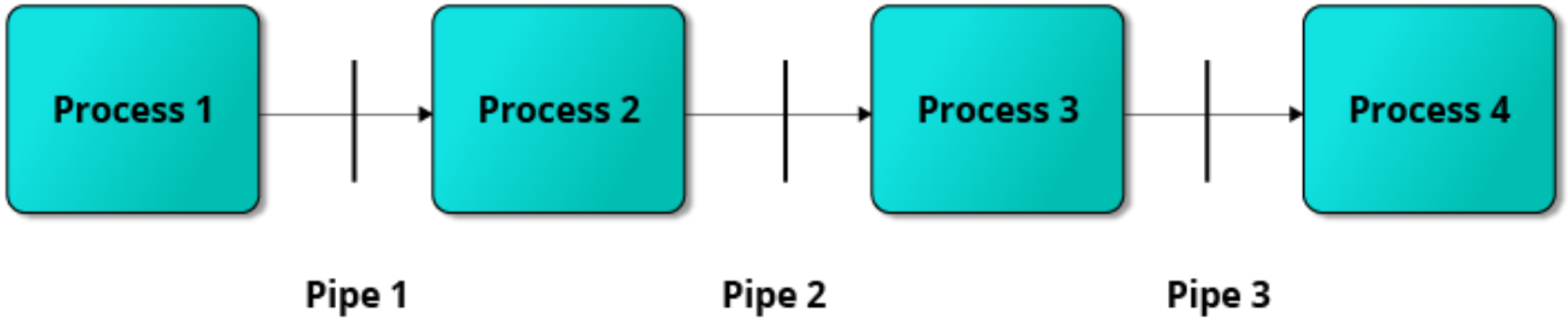
$ program > all-output-file 2>&1

$ program >& all-output-file
```

Pipes

```
$ command1 | command2 | command3
```

- This is extraordinarily efficient because `command2` and `command3` do not have to wait for the previous pipeline commands to complete before they can begin processing at the data in their input streams
- On multiple CPU or core systems, the available computing power is much better utilized and things get done quicker.



locate

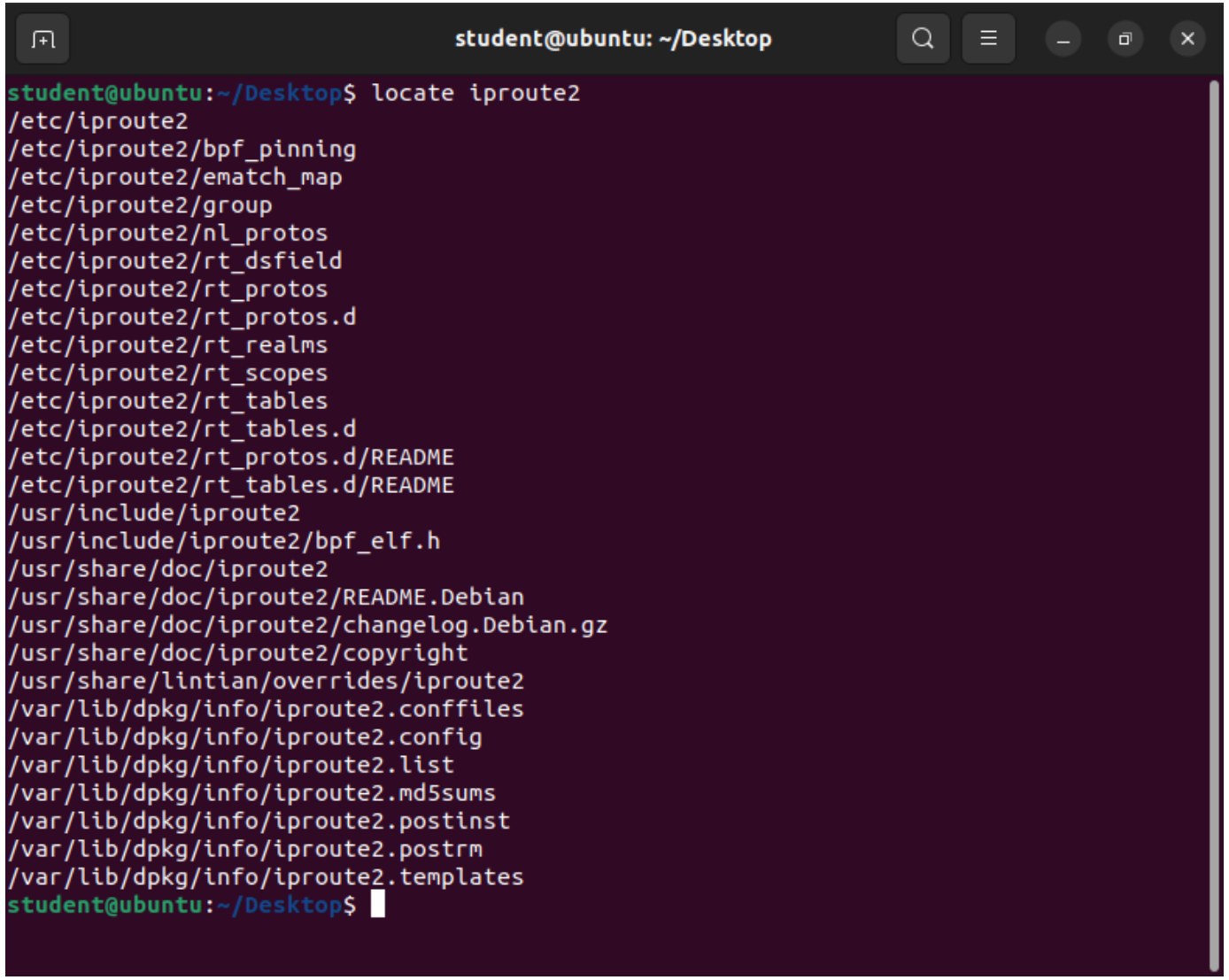
- The `locate` utility program performs a search while taking advantage of a previously constructed database of files and directories on your system, matching all entries that contain a specified character string. This can sometimes result in a very long list.
- To get a shorter (and possibly more relevant) list, we can use the `grep` program as a filter. `grep` will print only the lines that contain one or more specified strings, as in:

```
$ locate zip | grep bin
```

  - which will list all the files and directories with both zip and bin in their name. We will cover grep in more detail later. Notice the use of | to pipe the two commands together.

- `locate` utilizes a database created by a related utility, `updatedb`.
- Most Linux systems run this automatically once a day. However, you can update it at any time by just running `updatedb` from the command line as the root user.

`locate` Example



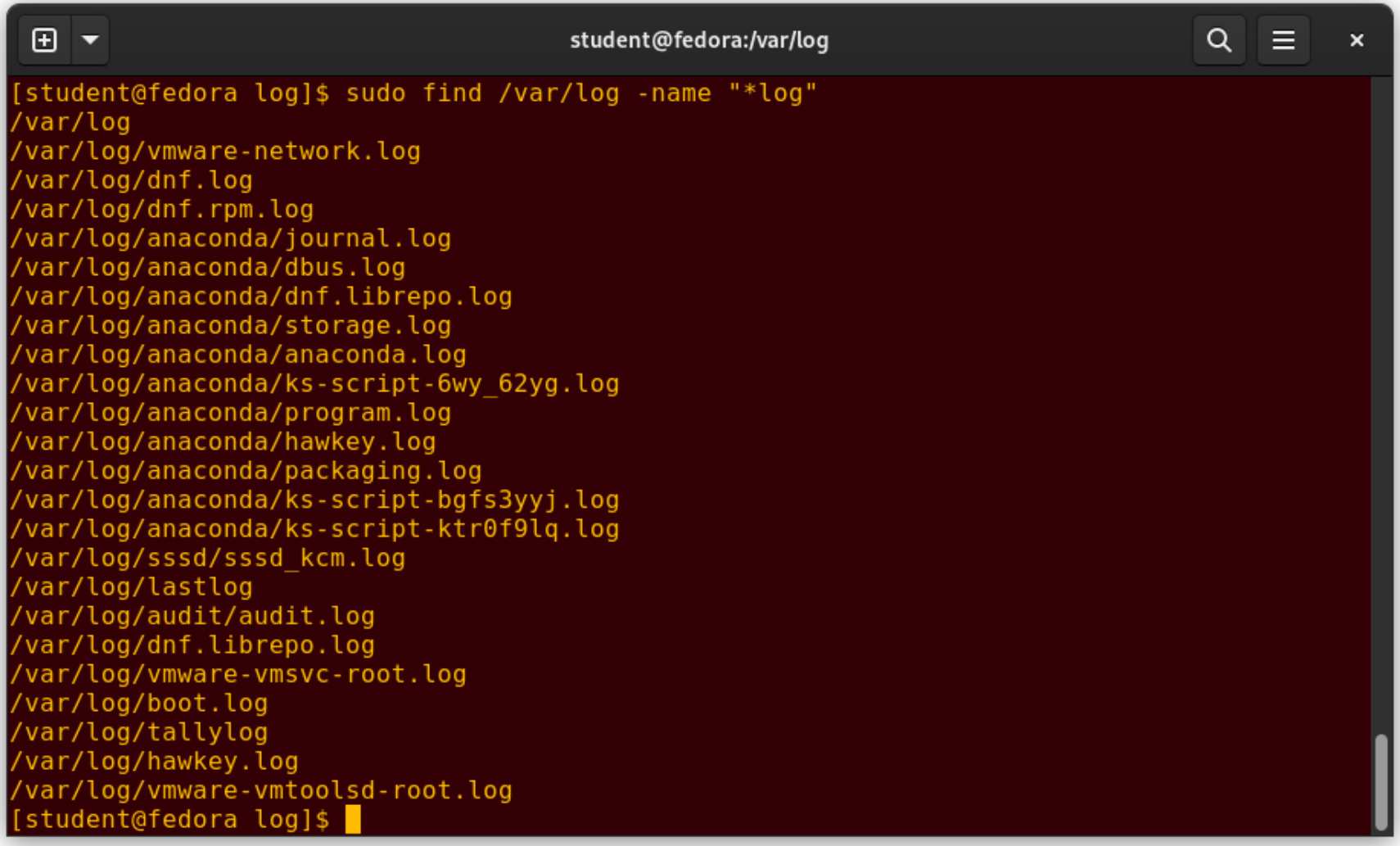
Searching for Files

```
$ ls ba?.out
$ ls *.out
$ find /usr -name gcc
$ find /usr -type d -name gcc
```

```
$ find /usr -type f -name gcc
$ find -name “*.swp” -exec rm {} ‘;’
$ find / -ctime 3
$ find / -size +10M –exec command {} ‘;’
```

Wildcard	Result
?	Matches any single character
*	Matches any string of characters
[set]	Matches any character in the set of characters, for example [adf] will match any occurrence of a, d, or f
[!set]	Matches any character not in the set of characters

Example of Searching for Files



Finding Files Example

